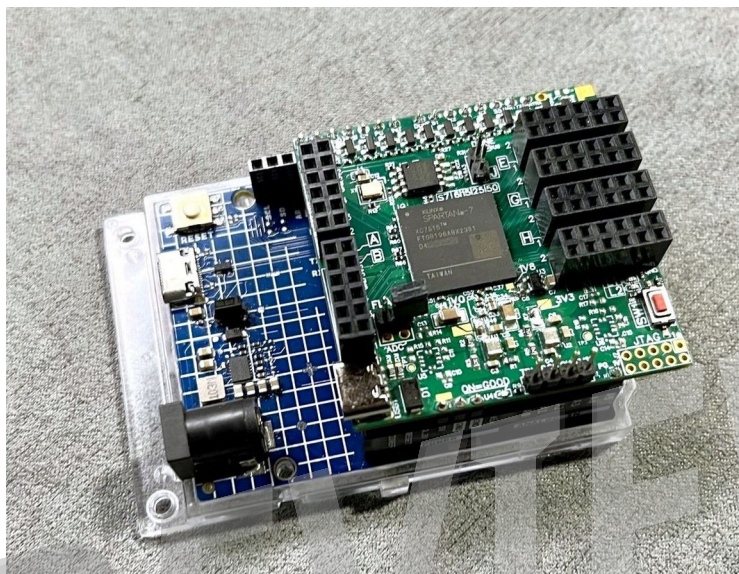


Corso MODULO FPGA

con introduzione a VHDL & Vivado



Indice generale

Capitolo 1 Introduzione.....	5
Capitolo 2 Cos'è una FPGA.....	8
Capitolo 3 Cos'è il VHDL.....	11
Capitolo 4 VHDL.....	13
4.1 VHDL Keywords.....	13
4.2 VHDL Libraries.....	16
4.3 VHDL Entities & Architecture.....	19
4.4 VHDL Generic.....	23
4.5 VHDL Data Types.....	26
4.6 VHDL Integer.....	30
4.7 When-Else Statements.....	32
4.8 Concurrent vs Sequential Statements.....	34
Capitolo 5 Progetto: porta AND.....	37
5.1 Creazione del progetto in Vivado.....	38
5.2 Codice VHDL.....	42
5.3 Assegnazione pin.....	43
5.4 Generazione bitstream.....	44
5.5 Upload bitstream.....	46
5.6 Collaudo.....	49
5.7 Generiamo tutte le porte logiche [Approfondimento].....	50
Capitolo 6 Simulazione VHDL.....	54
6.1 Test Bench porta AND.....	55
6.2 Vivado Simulator introduzione.....	59

6.3 Simulare partendo dal testbench.....	60
6.4 4-bit Shift Register Design.....	65
6.4.1 Test bench per il 4-bit shift register.....	69
6.4.2 Simulazione con Vivado.....	72
6.4.3 Come si realizza un Shift register, D Flip-Flop.....	73
6.4.4 Metastabilità e tempo di propagazione.....	74
6.4.5 Come Vivado ha implementato lo Shift register.....	75
6.4.6 Simulare anche gli imperment temporali[Approfondimento].....	78
6.4.6.1 Metastabilità setup time.....	81
Capitolo 7 VHDL Concetti base.....	82
7.1 Signals & Constants.....	82
7.2 VHDL Process.....	83
7.3 VHDL If Statements.....	84
7.3.1 If Statement Priority.....	84
Capitolo 8 Blink.....	85
8.1 Assegnazione pin.....	91
8.2 Codice completo BLINK.vhd.....	92
8.3 Display Binary counter.....	94
8.3.1 Codice BLINK_binary_cnt.vhd.....	96
Capitolo 9 Introduzione al Block Design.....	98
9.1 MUX 2bit.....	102
Capitolo 10 D Flip Flop.....	106
10.1 DFF.....	107
10.1.1 DFF con RESET SYNC & Clock EN.....	108
10.1.2 DFF primitiva "FDRE" [Approfondimento].....	113
10.2 Divisore di frequenza DIV2.....	117
10.2.1 T flip-flop.....	121
10.3 Divisore di frequenza DIV4.....	125
10.3.1 Codice "DIV4.vhd".....	127
10.4 Contatore asincrono & Divisore 8.....	130
10.4.1 Simulazione.....	137
10.4.2 Misure.....	140
10.4.3 Divisori / Contatori Asincroni.....	141
10.4.4 [Approfondimento] Misuriamo il ritardo di propagazione dei DFF.....	142
10.5 Divisore di frequenza DIV3 "ring-counter".....	143
10.5.1 Block Design DIV3.....	144
10.5.2 Ring-counter VHDL parametrico.....	145
10.6 Esercizio Toggle LED sul fronte di discesa.....	149
10.7 Rivedere, Capire & Approfondire.....	151
Capitolo 11 Anti-Debouncing & Top level.....	153
11.1 Codice VHDL "Debounce_btn.vhd".....	154
11.2 Test bench "test_Debounce_btn.vhd".....	155
11.3 Top-level.....	158
11.4 Rivedere, Capire & Approfondire.....	161
11.4.1 Test-bench automatico.....	161
11.4.2 MUX.....	164
11.4.3 DE-MUX.....	165
11.4.4 Shift register.....	166
11.4.5 JTAG.....	168
11.4.6 DEBUG ILA.....	172
11.4.6.1 Aggiungere ILA nel top-level.....	173

11.4.6.2 Aggiungere ILA in "Block Design"	179
11.4.6.3 Aggiungere ILA probe nella sintesi.....	181
11.4.7 DEBUG VIO.....	183
11.4.8 Clock Divider.....	187
11.4.8.1 CLOCK DIVIDER + ILA.....	190
Capitolo 12 PWM.....	191
12.1 Codice VHDL "pwm.vhd"	192
12.2 Test bench "test_pwm.vhd"	196
12.3 In Laboratorio: PWM LED.....	198
12.4 In Laboratorio: PWM LED & VIO.....	200
Capitolo 13 VHDL Generate Statement.....	202
13.1 Generate "for".....	203
13.2 Generate "if".....	205
13.3 Rivedere, Capire & Approfondire.....	206
13.3.1 Attributi KEEP e DONT_TOUCH in Vivado.....	206
13.3.2 Linea di ritardo con porte NOT.....	206
Capitolo 14 LFSR.....	208
14.1 3-Bit LFSR "LFSR_3.vhd"	210
14.2 5-Bit LFSR "LFSR_5.vhd"	211
14.3 5-Bit LFSR "Checker".....	212
14.4 Blink LED con LFSR.....	216
14.5 X-Bit LFSR.....	219
14.5.1 TOP-LEVEL X-LFSR.....	221
14.6 Rivedere, Capire & Approfondire.....	224
14.6.1 Blink LED.....	224
Capitolo 15 UART.....	226
15.1 Modulo UART TX.....	227
15.1.1 In Laboratorio:Test.....	230
15.2 VHDL: Array.....	232
15.3 Modulo UART TX "message".....	233
15.3.1 In Laboratorio:Test.....	238
15.4 VHDL: Funzioni.....	239
15.5 VHDL: Variabili.....	241
15.6 Modulo UART TX "status"	242
15.6.1 In Laboratorio:Test.....	250
15.7 Modulo UART RX.....	251
15.7.1 In Laboratorio:Test.....	254
15.8 Modulo UART RX "status"	256
15.8.1 In Laboratorio:Test.....	262
15.9 Uso pratico della UART : lettura e pilotaggio dei pin PMOD.....	263
15.9.1 In Laboratorio: TEST dei pin.....	265
15.10 Modulo UART "echo".....	268
15.10.1 In Laboratorio: verifica echo da terminal.....	274
15.11 Rivedere, Capire & Approfondire.....	275
15.11.1 Modulo UART TX "status" V2.0.....	275
15.11.2 Top-level UART TX & RX status.....	280
15.11.3 Come salvare gli eventi: "alarm_reg.vhd".....	282
15.11.3.1 Uso alarm_reg nell'interfaccia registri FPGA.....	285
Capitolo 16 Softcore.....	289
16.1 Primo progetto con softcore: UART TX.....	290
16.1.1 In Laboratorio: verifica UART da terminale.....	295

16.2 Progetto con Softcore: UART RX char.....	296
16.2.1 In Laboratorio: verifica UART RX.....	297
16.3 Progetto con softcore: UART RX string.....	298
16.3.1 In Laboratorio: verifica UART RX.....	299
16.4 Progetto con softcore: GPO.....	300
16.4.1 In Laboratorio: verifica GPO.....	302
16.5 Progetto con Softcore: GPI.....	303
16.5.1 In Laboratorio: verifica GPI.....	305
16.6 Progetto con Softcore: UART string compare.....	306
16.6.1 In Laboratorio: Comandi tramite comandi seriali.....	307
Capitolo 17 Appendice.....	308
17.1 Revisioni hardware.....	308
17.2 PIN-OUT.....	309
17.3 Schema a blocchi.....	310

Capitolo 1

Introduzione

Il "MODULO FPGA SPARTAN7" è una scheda di sviluppo dotata di FPGA e una memoria SPI NOR FLASH (SNOR).

Il modulo può essere usato in due modalità:

- In abbinamento con Arduino R4, che funge da programmatore, interfaccia di gestione ecc...
- In modalità stand-alone, ma è necessario caricare prima il file di configurazione FPGA nella memoria SNOR, successivamente basta alimentarlo dalla sua porta USB-C senza più la necessità di Arduino.

Come sappiamo la FPGA non ha una memoria non-volatile nel quale è possibile scrivere il "file di configurazione" ma dispone solo di una "memoria volatile", quindi necessita una esterna nel quale andremo a salvare il file di configurazione detta SNOR FLASH.

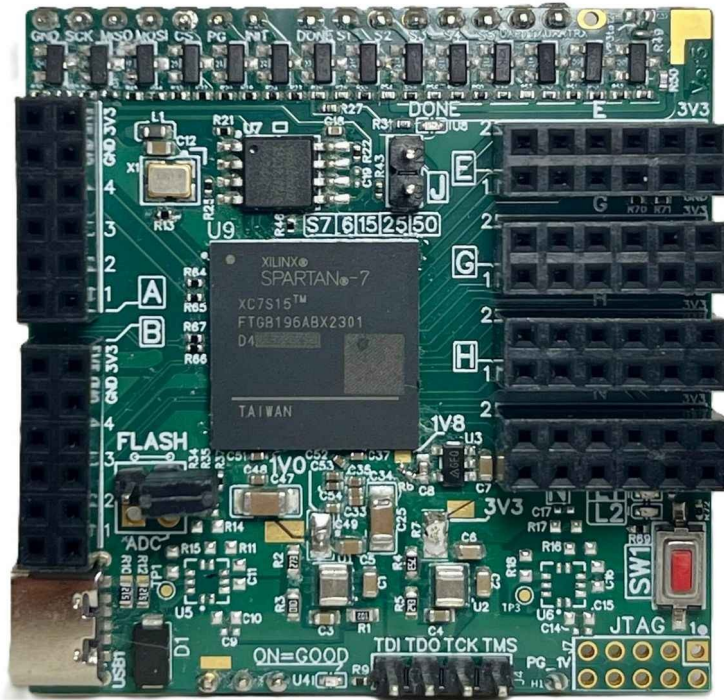
L'Arduino R4 viene impiegato per:

- Configurare la FPGA caricando il file *bitstream* direttamente (configurazione persa al reset).
- Scrivere il file binario di configurazione FPGA nella memoria SNOR, consentendo poi il funzionamento stand-alone del modulo.
- Comunicare con la FPGA tramite interfaccia **UART**.
- Monitorare lo stato della FPGA ed eseguire operazioni di reset.
- Interfacciarsi con 5 linee di I/O a 5V dedicate (S1–S5), mappate come segue:
 - S1 → PIN2
 - S2 → PIN3
 - S3 → PIN4
 - S4 → PIN5
 - S5 → PIN6

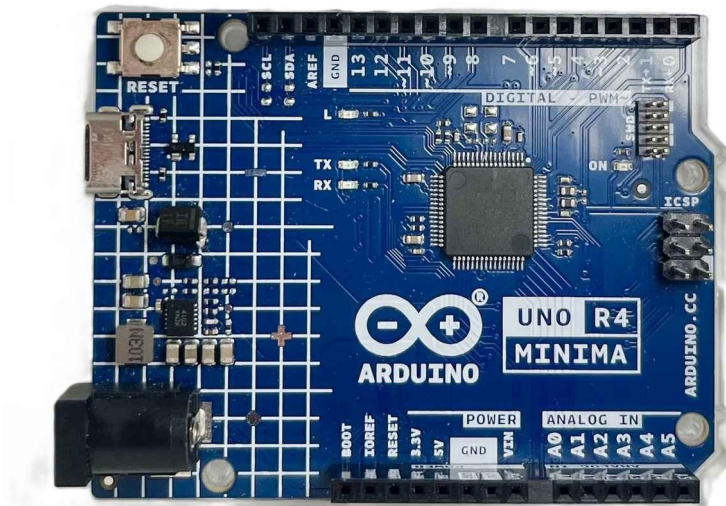
Sull'Arduino R4 viene caricato un software dedicato alla gestione del modulo FPGA.

Cosa Serve:

- MODULO FPGA SPARTAN7
 - PCB Version=3[min Rev00 vedi qui [18.1](#)]
 - Pin-out [18.2](#)
 - System overview [18.3](#)



- Arduino UNO R4 Minima o Arduino UNO R4 WiFi



- Arduino® IDE 2.3.0 :
<https://www.arduino.cc/en/software/>

- Software per Arduino [FPGA_Schield-cli.ino Ver 0.0]
https://github.com/Jampag/FPGA-Shield-Arduino-compatible/blob/main/software/FPGA_Schield-cli.ino
- Tera Term [Version 5.4.0]
<https://teratermproject.github.io/>
<https://github.com/TeraTermProject/teraterm/releases>
- File pinout del "MODULO FPGA SPARTAN7" :
https://github.com/Jampag/FPGA-Shield-Arduino-compatible/blob/main/pinout/pinout_ver003_rev00.xdc
- Vivado [Referance version 2024.2]
- Cavo USB-C (>=USB 2.0).
- Guida Rapida introduttiva: consiglio la lettura della guida prima di affrontare questo corso, in quanto è ricco di descrizione e dettagli sul "MODULO FPGA SPARTAN7" e sul come configurare il banco di sviluppo.
<https://github.com/Jampag/FPGA-Shield-Arduino-compatible/blob/main/documents/Quick-start-GR01.pdf>

Questo progetto non è affiliato né approvato da Arduino.

Capitolo 2

Cos'è una FPGA

Cos'è una FPGA?

È un dispositivo programmabile, nel quale si configurano una matrice di celle logiche, queste celle possono sia essere collegate tra di loro che verso i pin di uscita.

Cosa si intende per programmare allora?

Definire le funzioni logiche di ogni cella e definire il loro collegamento, in modo da realizzare un circuito logico arbitrario. Il vantaggio è che non avendo un processore tutti i "mattoncini" che descritti sono autonomi e paralleli.

Quali sono le applicazioni?

-Per il calcolo parallelo, ad esempio: elaborazione video, compressione hardware di dati, crittografia ecc..

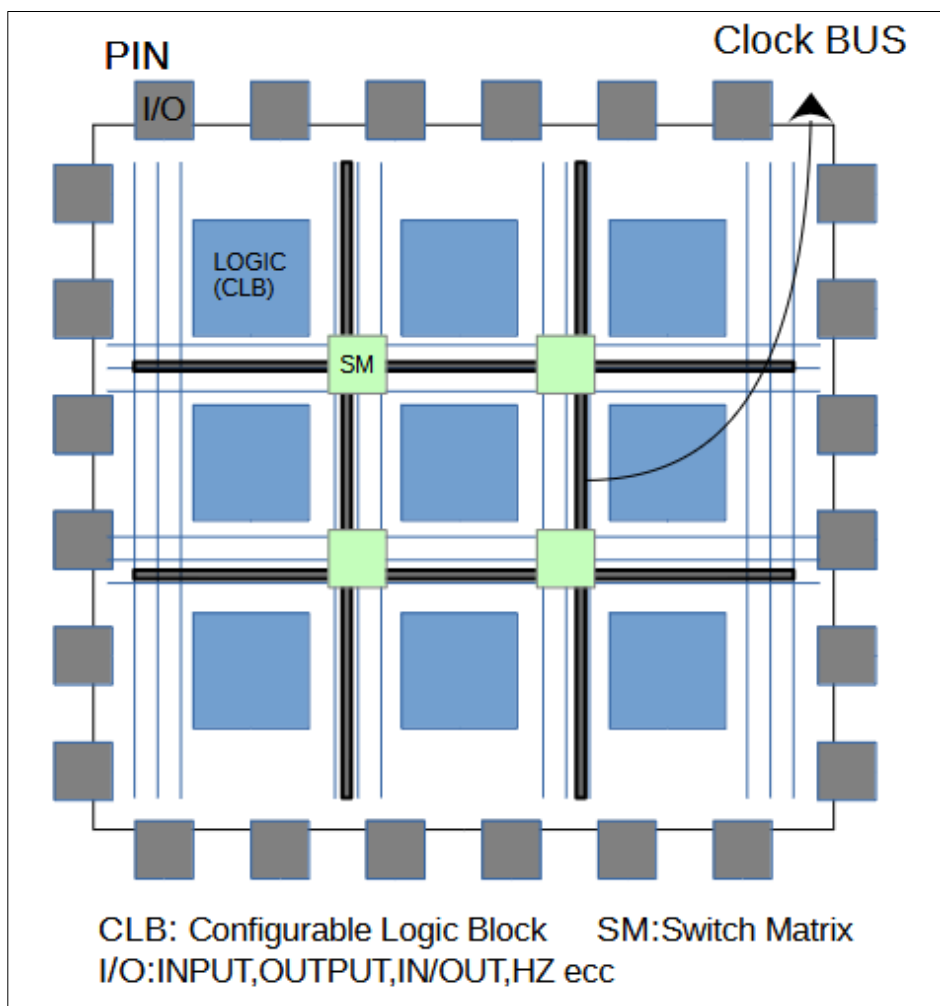
-Conversione di segnali digitali: da parallelo a seriale, conversione di tensione, convertitore di protocollo (es da SPI a I2C), I/O expander, ecc ...

-È possibile realizzare controllori HW dei vari protocolli seriali UART, SPI, I2C, ETH ecc... senza essere vincolati alle risorse di un microcontrollore.

-Emulazione di hardware, avendo a disposizione migliaia di logica base è possibile realizzare addirittura un microprocessore, una console di gioco, componenti digitali ecc...

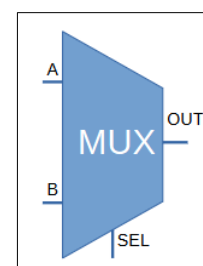
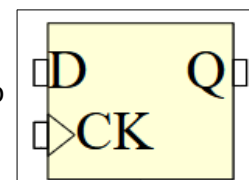
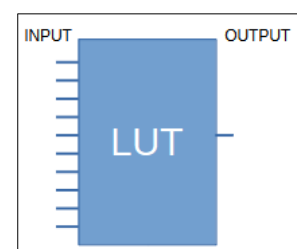
La FPGA è volatile, perde la configurazione in mancanza di alimentazione, quindi vengono associata a delle memorie non volatili come le SNOR che ad ogni riavvio si auto-configurano.

Schema di una FPGA



CLB: È cuore della FPGA formata da: LUT, FF e MUX

- **LUT (Look-up table):**
è una logica combinatoria configurabile. Per logica combinatoria si intende che lo stato di uscita è istantaneamente dipendente dagli stati di ingresso, come le porte logiche. In questo caso è una porta logica universale dove si descrive la tabella della verità.
- **Flip-Flop:** è un elemento di memoria di un bit (anche detto registro), l'uscita dipende sia dagli ingressi che dal clock, può rappresentare logica sequenziale.
- **Multiplexer:** è come un deviatore, l'uscita è A o B dipende dallo stato del SEL (select).

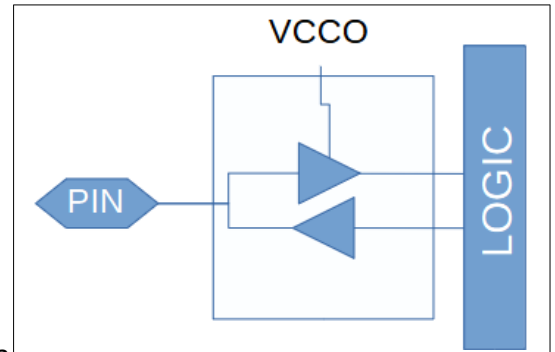


SM: È una matrice di fili che possono essere collegati tra di loro con interruttori, realizzati con semiconduttori di solito transistor.

I segnali di clock hanno una matrice ridotta ma dedicata, necessaria per la corretta sincronizzazione dei flip-flop, avente bassa latenza.

I/O: È l'interfaccia con il mondo esterno

- Può essere configurata come INPUT
- Può essere configurata come OUTPUT
- Può essere configurata come BIDIR
- Può essere configurata come tri-state o open-collector
- Si possono definire lo standard elettrico LVCMOS3V3, LVCMOS1V8, LVCMOS0V9 ...
- Definire lo standard anche differenziale (LVDS, HSTL...)
- Si definisce anche la corrente di uscita 2,4,6,8,12,16 mA
- Si possono istanziare Pull-up a volte anche pull-down, terminazioni 50Ω, 100Ω



Capitolo 3

Cos'è il VHDL

Benvenuto

In questo capitolo presenteremo cos'è VHDL, a cosa serve e perché è uno strumento fondamentale per l'ingegneria elettronica digitale moderna.

Che cos'è VHDL?

VHDL è un *linguaggio di descrizione dell'hardware* (Hardware Description Language – HDL).

Il suo scopo non è quello di scrivere software, bensì di descrivere circuiti digitali.

È molto simile a Verilog, un altro HDL molto diffuso: entrambi svolgono la stessa funzione, cioè permettono di definire il comportamento e l'architettura di un circuito digitale.

➤ È importante chiarire un concetto: VHDL non è un linguaggio di programmazione tradizionale come C, Java ecc..

Con questi linguaggi le istruzioni vengono eseguite in modo sequenziale, una dopo l'altra.

In VHDL, invece, non scriviamo software ma descriviamo hardware, cioè la struttura e il funzionamento di un circuito logico.

Pensare in termini di hardware

Utilizzare VHDL significa cambiare prospettiva: non stiamo più sviluppando codice da eseguire, ma stiamo **progettando uno schema digitale**.

In pratica con VHDL possiamo definire:

- una porta logica AND, OR XOR
- un comparatore
- un Flip-Flop
- un contatore
- oppure un qualsiasi altro blocco logico digitale.

In questo senso, VHDL permette di **architettare un circuito digitale** attraverso una descrizione testuale che potrà poi essere sintetizzata su un dispositivo hardware, come un **FPGA** o un **CPLD**.